If you now lock *another* `hierarchical_mutex` while holding the lock on this first one, the value of `this_thread_hierarchy_value` reflects the hierarchy value of the first mutex. The hierarchy value of this second mutex must now be less than that of the mutex already held in order for the check ❷ to pass.

Now, it's important to save the previous value of the hierarchy value for the current thread so you can restore it in `unlock()` ❻; otherwise you'd never be able to lock a mutex with a higher hierarchy value again, even if the thread didn't hold any locks. Because you store this previous hierarchy value only when you hold the `internal_mutex` ❸, and you restore it *before* you unlock the internal mutex ❻, you can safely store it in the `hierarchical_mutex` itself, because it's safely protected by the lock on the internal mutex.

`try_lock()` works the same as `lock()` except that if the call to `try_lock()` on the `internal_mutex` fails ❼, then you don't own the lock, so you don't update the hierarchy value and return `false` rather than `true`.

Although detection is a runtime check, it's at least not timing dependent—you don't have to wait around for the rare conditions that cause deadlock to show up. Also, the design process required to divide the application and mutexes in this way can help eliminate many possible causes of deadlock before they even get written. It might be worth performing the design exercise even if you then don't go as far as actually writing the runtime checks.

Extending These Guidelines Beyond Locks

As I mentioned back at the beginning of this section, deadlock doesn't just occur with locks; it can occur with any synchronization construct that can lead to a wait cycle. It's therefore worth extending these guidelines to cover those cases too. For example, just as you should avoid acquiring nested locks if possible, it's a bad idea to wait for a thread while holding a lock, because that thread might need to acquire the lock in order to proceed.